



温州肯恩大学

WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01

Python Programming for Business

Python Data Structure and Control Flow

Dawei Chen

Week 3-1, 10 March 2026

Today's Learning Objectives



By the end of this session, you should be able to:

- Understand Python's core **data structures**: list, tuple, set, dictionary
- Select appropriate data structures for different tasks
- Use **conditional statements** and **loops** to control program flow
- Apply these tools to basic business analytics tasks

Python Data Structure Overview

- Basic data types: `int`, `float`, `str`, `bool`
- Built-in data structures: `list`, `tuple`, `set`, `dict`

Data Structure	Ordered	Mutable	Allow Duplicates
List 列表	Yes	Yes	Yes
Tuple 元组	Yes	No	Yes
Set 集合	No	Yes	No
Dictionary 字典	Yes	Yes	Key: No Values: Yes

- **Ordered:** elements are stored and accessed in a specific, consistent sequence (their position is preserved).
- **Mutable:** contents of an object can be modified after creation without creating a new object.

Lists 列表

- Stores multiple items in a single variable
- Written within square brackets []
- Properties
 - **Ordered:** elements keep their position
 - **Mutable:** elements can be modified
 - Allows **duplicates**
 - Can store **mixed data types**

Lists are the one of the most commonly used data structured in data analysis.

```
thislist = [1, 'Wenzhou', 3.14, True, 'Wenzhou']
print(type(thislist))
thislist[0]

[1] ✓ 0.0s
... <class 'list'>
... 1

thislist[1] = 'Hangzhou'
thislist

[2] ✓ 0.0s
... [1, 'Hangzhou', 3.14, True, 'Wenzhou']

thislist2 = [1, 'Wenzhou', 3.14, True, 'Hangzhou']
thislist == thislist2

[3] ✓ 0.0s
... False
```

List Operations

Exercises

- Indexing and slicing
- Change, add, remove items
- Sort, copy, join lists
- Loop through a list
- List comprehensions
- Built-in list methods

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []

for x in fruits:
    if "a" in x:
        newlist.append(x)

print(newlist)
```

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits if "a" in x]

print(newlist)
```

Tuples 元组

- Stores multiple items in a single variable
- Written within **round** brackets ()
- Properties
 - Ordered: elements keep their position
 - **Immutable**: elements **cannot** be modified
 - Allows duplicates
 - Can store mixed data types

```
thistuple = (1, 'Wenzhou', 3.14, True, 'Wenzhou')
print(type(thistuple))
thistuple[0]
✓ 0.0s

<class 'tuple'>
1

thistuple2 = (3.14, 'Wenzhou', 1, True, 'Wenzhou')
thistuple == thistuple2
✓ 0.0s

False

thistuple[1] = 'Hangzhou'
thislist
✗ 0.2s

-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 thistuple[1] = 'Hangzhou'
      2 thislist

TypeError: 'tuple' object does not support item assignment
```

Tuple Operations

Exercises

- Indexing and slicing
- *Change, add, remove items?*
- Unpack, join, multiply tuples
- Loop through a tuple
- Built-in tuple methods

Lists

vs.

Tuples

Ordered

Ordered

Mutable

Immutable

Suitable for data
that may change
over time

- Suitable for fixed, stable records
- Prevent accidental modification
- Signal intentional immutability
- More memory-efficient

Process datasets •
Compute statistics •
Clean data •

- Coordinates
- Database records
- Configuration settings

Sets 集合

- Stores multiple items in a single variable
- Written within curly brackets { }
- Properties
 - **Unordered**
 - **Unindexed:** cannot access items by an index, but can loop or in
 - **Unchangeable**, but can add / remove
 - **Not allow duplicates**
 - Can store mixed data types

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
    print(x)

print("banana" in thisset)

print("banana" not in thisset)
```

✓ 0.0s

apple
cherry
banana
True
False

Set Operations

Exercises

- Access, add, remove set items
 - `add()`, `remove()`, `pop()`, `discard()`
- [Join sets](#)
 - `union()`, `update()`, `intersection()`, `difference()`, `symmetric_difference()`
- [Built-in set methods](#)

Useful for:

- Customer segmentation
- Duplicate removal
- Data cleaning

Dictionary 字典

```
thisdict = {  
    "brand": "Porsche",  
    "model": "Panamera",  
    "year": 2024  
}
```

- Stores data values in a **key:value** pairs
- Written within curly brackets `{ }`, and have keys and values
- Properties
 - Ordered: items have a defined order that will not change, indexing
 - Changeable
 - Not allow duplicate: keys must be unique, values can repeat
 - Keys must be **immutable** and **hashable**
 - Values can be of any data type

Dictionary Operations

Exercises

- Access, change, add, remove items
- [Loop](#) dictionaries
- Copy dictionaries
- Nested dictionaries
- [Built-in dictionary methods](#)

```
thisdict = {  
    "brand": "Porsche",  
    "model": "Panamera",  
    "year": 2024  
}
```

```
for x in thisdict:  
    print(x)
```

```
for x in thisdict:  
    print(thisdict[x])
```

```
for x in thisdict.values():  
    print(x)
```

```
for x in thisdict.keys():  
    print(x)
```

```
for x, y in thisdict.items():  
    print(x, y)
```

Summary of Python Data Structures

Data Structure	Ordered	Mutable	Allow Duplicates	Typical Use
List 列表	Yes	Yes	Yes	Store and modify an ordered collection of items
Tuple 元组	Yes	No	Yes	Represent fixed and stable groups of values
Set 集合	No	Yes	No	Keep only unique items and support set operations
Dictionary 字典	Yes	Yes	Keys: No Values: Yes	Map keys to values for efficient access

flexible sequence → list

fixed sequence → tuple

unique values → set

key-value mapping → dictionary

Control Flow

- Control flow **determines the order** in which a program executes instructions.
- Without control flow, programs execute **line by line from top to bottom**.
- With control flow, programs can
 - Make decisions: `if`, `elif`, `else`
 - Repeat tasks: `for`, `while`, `loops`
 - Handle different conditions in dynamic situations
- Key idea
 - Control flow allows programs to become dynamic, flexible, and responsive to data

If ... else

`if` statement executes code when a condition is true.

```
sales = 200

if sales > 150:
    print("High sales")
else:
    print("Normal sales")
```

```
sales = 200

if sales > 150:
    print("High demand")
else:
    print("Normal demand")
```

```
sales = 180

if sales > 200:
    print("Very high sales")
elif sales > 150:
    print("High sales")
else:
    print("Normal sales")
```

```
username = "MGS3001"
password = False
is_active = True

if username:
    if password:
        if is_active:
            print("Login successful")
        else:
            print("Account is not active")
    else:
        print("Password required")
else:
    print("Username required")
```

Nested If

Guess the output

For Loops

for loops

- Iterates over a sequence (e.g., list, tuple, set, dictionary, string)
- **break**: stops the loop before all items are processed
- **continue**: skips the current iteration and move to the next
- **pass**: an **if** statement cannot be empty, if no action is required, use **pass** as a placeholder to avoid an error (*why? when?*)

range() function

- Generates a sequence of numbers
- Starts from 0 and increments by 1 (by default) (will revisit after introducing function)
- Stops before the specified end value

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

```
age = 16

if age < 18:
    pass # TODO: Add underage logic later
else:
    print("Access granted")
```

```
for i in range(6):
    print(i)
```

While Loops

while loops

- Execute a set of statements **as long as a condition is true**
- **break**: stops the loop even if the while condition is true
- **continue**: stops the current iteration and continue with the next
- **else**: run a block of code once when the condition no longer is true

Guess the output

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```


In-Class Exercise

- Context:

The following code attempts to compute **total spending per customer**, but it contains a logical error.

- Tasks:

- Predict the output of the current program.
- Revise the code so that it correctly **aggregates spending per customer**.

- Expected Output (correct result):

`{'Alice':1280, 'Bob':325, 'David':1200}`

```
transactions = [
    ("Alice", "Laptop", 1200),
    ("Bob", "Mouse", 25),
    ("Alice", "Keyboard", 80),
    ("David", "Laptop", 1200),
    ("Bob", "Monitor", 300)
]

spending = {}

for t in transactions:
    customer = t[0]
    price = t[2]

    if customer not in spending:
        spending[customer] = price
    else:
        spending[customer] = price

print(spending)
```

Summary

- Python Data Structures
 - **List** → ordered and mutable collection
 - **Tuple** → ordered but immutable record
 - **Set** → unordered collection of unique elements
 - **Dictionary** → key-value mapping for structured data
- Control Flow in Python
 - **Decision making** → `if`, `elif`, `else`
 - **Iteration** → `for`, `while`
 - **Loop control** → `break`, `continue`, `pass`



- Next Class: Functions – modularizing code and reusing logic
- Assignment 1 Due: 23:59, Sunday, 15 March

Q&A

CBPM B214

dchen@kean.edu

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30